

9.3. Указатель на функцию. Динамическое связывание

Указатель на функцию - переменная, которая содержит адрес некоторой функции. Соответственно, косвенное обращение по этому указателю представляет собой вызов функции.

Определение указателя на функцию имеет вид:

```
int    (*pf)();           // без контроля параметров вызова
int    (*pf)(void);       // без параметров, с контролем по прототипу
int    (*pf)(int, char*); // с контролем по прототипу
```

В соответствии с принципом контекстного определения типа данных эту конструкцию следует понимать так: **pf** - переменная, при косвенном обращении к которой получается функция с соответствующим прототипом, например **int_F(int, char*)**, то есть **pf** содержит адрес функции или указатель на функцию. Следует обратить внимание на то, что в определении указателя присутствует прототип - указатель ссылается не на произвольную функцию, а только на одну из функций с заданной схемой формальных параметров и результата.

Перед началом работы с указателем его необходимо назначить на соответствующий объект, в данном случае - на функцию. В синтаксисе Си выражение вида **&имя_функции** имеет смысл - начальный адрес функции или указатель на функцию. Кроме того, по аналогии с именем массива использование имени функции без скобок также интерпретируется как указатель на эту функцию. Указатель может быть инициализирован и при определении.

```
int    INC(int a) { return a+1; }
extern int DEC(int);
int    (*pf)(int);
pf = &INC;
pf = INC;           // присваивание указателя
int    (*pp)(int) = &DEC; // инициализация указателя
```

Естественно, что функция, на которую формируется указатель, должна быть известна транслятору - определена или объявлена как внешняя. Синтаксис вызова функции по указателю совпадает с синтаксисом ее определения.

```
n = (*pf)(1) + (*pp)(n); // эквивалентно
n = INC(1) + DEC(n);
```

Как и обычный указатель, указатель на функцию имеет размерность адреса, т.е. на уровне архитектуры является машинным словом, содержащим адрес функции в соответствии с системой формирования адресов процессором (системой адресации памяти). Это позволяет, в свою очередь, опускаться до таких машинно-зависимых действий как вызов функции по заданному адресу памяти, преобразуя целую константу к указателю на функцию.

```
void (*pf)(void) = (void*)(void)0x1000; // Константа 1000 – шестнадцатеричный адрес
void main() { (*pf)(); }                 // функции, которую вызывает main
                                         // void*(void) – абстрактный тип данных –
                                         // указатель на функцию
```



Указатель на функцию прежде всего имеет отношение к системному программированию. Достаточно сказать, что кроме Си в явном виде он встречается только в Паскале (процедурный тип), но во многих случаях входит в

скрытые механизмы управления программной средой, известные как **динамическое связывание функций**. Динамическое и статическое связывание мы уже обсуждали в 5.6., но там это относилось к переменным. По отношению к функции термин динамическое связывание следует понимать как установление соответствия между именем функции и ее адресом. Если рассматривать этот вопрос по отношению обычным функциям на Си, то можно выделить три этапа связывания:

- если определение функции и ее вызов находятся в одном модуле (файле), то транслятору ничего не мешает вычислить относительный адрес функций внутри объектного модуля во время трансляции – **статическое связывание**;
- при вызове внешней функции, определение которой находится в другом файле (а программный код – в другом объектном модуле), адрес функции становится известен при компоновке программного файла из объектных модулей. Хотя процедура компоновки выполняется после трансляции, данное связывание также называется **статическим**;
- набор внешних функций (объектный модуль целиком) может быть оформлен в виде динамически связываемой библиотеки – DLL (dynamic linking library). Она загружается в память одновременно с программным модулем и связывание имени внешней функции с ее адресом в адресном пространстве DLL происходит при загрузке. Это связывание уже считается **динамическим**. При наличии в каждом приложении отображаемого адресного пространства (виртуальной памяти) программный код DLL может одновременно использоваться (разделяться) несколькими приложениями.

Если на уровне среды программирования или непосредственно в Си-программах имеет место элемент динамического связывания, то с уверенностью можно сказать, что в том или ином виде (явно или неявно) используется указатель на функцию:

- уже упомянутые нами DLL, а также все возможные виды динамической загрузки внутреннего (исполняемого) программного кода;
- виртуальные функции в Си++ (см.12.4.) – при конструировании объекта производного класса в базовый класс помещается указатель на таблицу виртуальных функций для производного класса (массив указателей на функции);
- функции, работающие с произвольными типами данных – для настройки на конкретный тип данных в качестве «довеска» получают указатель на функцию, которая «знает», как с ним работать. В результате основной алгоритм изолируется от типа данных, с которым работает.

Таблицы функций, вызов по имени

Тип данных вида **void (*pp[])()** расшифровывается в соответствии с контекстным определением типа данных как массив указателей на функции с общим прототипом (схемой передачи параметров и результата) – последовательность операций в контексте – массив – указатель – вызов функции. Образно, хотя и не совсем точно этот тип можно назвать **таблицей функций**, вызов которых может производиться по номеру (индексу).

В самом простом примере наличие таблицы функций позволяет сделать вызов функции по заданному символьному имени регулярным (циклическим). В принципе, то же самое можно сделать с помощью обычного переключателя (switch), но для этого придется ее каждую новую функцию добавлять фрагмент программного кода. Здесь же достаточно внести в массивы символьное имя и ее адрес.

```

extern double sin(double);
extern double cos(double);
extern double tan(double);
char    *names[] = { "sin","cos","tan",NULL};    // Массив имен (указатели на строки)
double  (*pf[])(double) = { sin, cos, tan};      // Таблица функций (адреса функций)
//-----93-01.cpp
//---- Вызов функции по имени из заданного списка
double call_by_name(char *pn, double arg) {
for ( int i=0; names[i]!=NULL; i++)
    if (strcmp(names[i],pn) == 0) {              // Имя найдено -
        return ((*pf[i])(arg));                 // вызов функции по i-му
    }                                             // указателю в массиве pf
return 0.;}

```

В принципе, используя технику работы с двоичными файлами и динамические массивы, можно было бы выполнить динамическую загрузку функций из программных файлов. Однако эта задача является архитектурно зависимой и требует учета особенностей перемещения программного кода и защиты памяти программы в операционной системе.

Указатель на функцию как средство параметризации алгоритма

Оригинальность и обособленность указателя на функцию заключается в том, что указываемым объектом является не переменная (компонента данных программы), а функция (компонента алгоритма). Но сущность указателя при этом не меняется: если обычный указатель позволяет параметризовать алгоритм обработки данных, то указатель на функцию позволяет параметризовать сам алгоритм. Это значит, что некоторая его часть может быть заранее неизвестна (не определена, произвольна) и будет подключаться к основному алгоритму только в момент его выполнения (динамическое связывание).

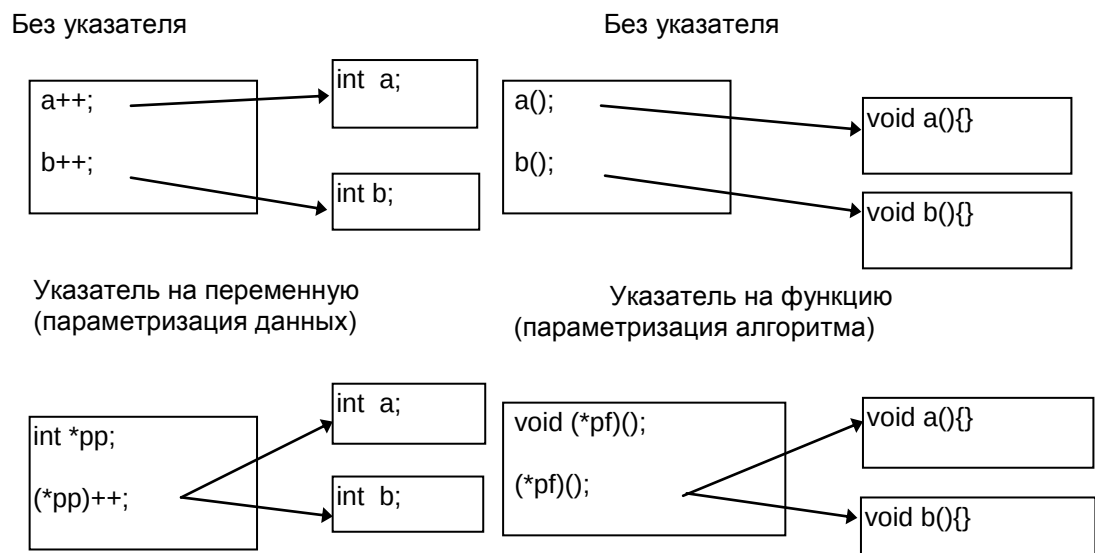


рис. 93-1. Параметризация алгоритма

Для реализации указанного принципа основная функция должна получать необходимый для ее работы «довесок» в виде формального параметра – указателя на функцию. В качестве примера приведем функцию вычисления определенного интеграла, в которой подынтегральная функция (в математическом смысле) передается в виде вычисляющей ее функции (в «программистском» смысле), которая передается формальным параметром - указателем на функцию.



«Функция, вычисляющая функцию, передается в виде указателя на функцию»

```
//-----93-02.cpp
//-----Численное интегрирование произвольной функции
double INTEG(double a, double b, int n, double(*pf)(double))
// a,b - границы интегрирования, n - число точек
// pf - подынтегральная функция
{ double s,h,x;
  for (s=0., x=a, h = (b-a)/n; x <=b; x+=h) s += (*pf)(x) * h;
  return s; }
extern double sin(double);
void main() { printf("sin(0..pi/2)=%f\n",INTEG(0.,M_PI/2,40,sin)); }
```

Другой частный пример подобного рода – сортировка одной и той же структуры данных по разным критериям. В массиве структур типа **user** для задания способа сравнения используется указатель на внешнюю функцию сравнения, которая в качестве параметров получает две ссылки на сравниваемые структуры.

```
//-----93-03.cpp
//-----Сортировка массива структур по разным критериям
struct user{
    char name[20];
    int account;
    double time;
} S[]={...};
//----- Вставка погружением
// cmp - указатель на функцию сравнения двух struct user
void sort(user A[], int (*cmp)(user&,user&)){
    for (int i=1;A[i].name[0]!=0;i++){
        for(int j=i; j>0 && (*cmp)(A[j],A[j-1])<0; j--){
            user c=A[j];
            A[j]=A[j-1]; // Обмен элементов массива -
            A[j-1]=c; } // структурированных переменных
        }
//----- функция сравнения по имени
int cmp_name(user &u1, user &u2){
    return strcmp(u1.name,u2.name); }
//----- функция сравнения по account
int cmp_account(user &u1, user &u2){
    return u1.account - u2.account; }
//----- функция сравнения по time
int cmp_time(user &u1, user &u2){
    return u1.time - u2.time; }

void main(){
    sort(S,cmp_name);
    sort(S,cmp_account);
    sort(S,cmp_time); }
```

Этот пример можно обобщить на случай использования данных любого типа. В Си отсутствует понятие произвольный (неопределенный) тип, но понятие указатель на неопределенный (произвольный) тип существует – это **void***. Однако прямое использование этого указателя невозможно. Какой же здесь выход?

Если существуют несколько идентичных структур данных, отличающихся типом хранимых данных, то один и тот же алгоритм будет отличаться только отдельными операциями, касающимися переменных этого типа. Например, при сортировке массива указателей на строки сравнение реализуется выражением вида **strcmp(p[i],p[i-1])**, а в массиве указателей на целые используется непосредственное сравнение ***p[i]<*p[i-1]**. Если эту операцию вынести за пределы алгоритма, реализовать отдельной функцией, а указатель на нее передавать в качестве параметра, то мы получим универсальную

функцию сортировки массивов указателей на переменные любого типа данных. Можно назвать ее условно (хотя и не совсем точно) **итератором**.

Типичными итераторами являются:

- итератор обхода (**foreach**), выполняющий для каждой переменной в структуре данных указанную функцию;
- итераторы проверки и поиска по условию (**firstthat**), возвращающие указатель на первую (последнюю, очередную) переменную, которая удовлетворяет условию, проверяемому в функции;
- итераторы, работающие на упорядоченных данных, использующие сравнение: сортировка, поиск минимального, двоичный поиск, включение с сохранением порядка.

Итератор являет собой пример универсализации алгоритма, основанный на разделении «компетенций» между различными его частями:

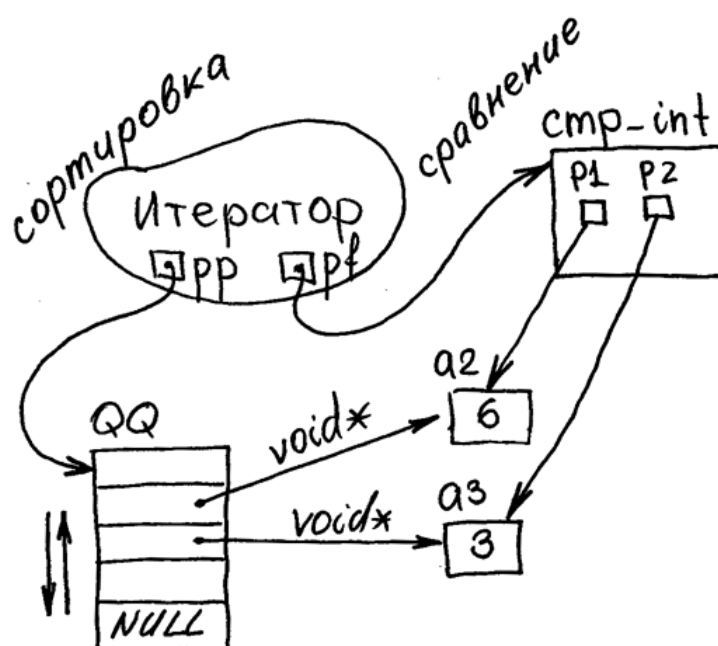


Рис. 93-2. Структура итератора

- структура данных, обрабатываемая итератором, содержит в своих элементах указатели на переменные произвольного (неизвестного для итератора) типа **void***, но одинакового в каждом экземпляре структуры данных. Структура данных «знает», где хранятся переменные, но «не знает», как их обработать;
- итератор получает в качестве параметров указатель на структуру данных и указатель на функцию обработки. Эта функция «знает», как обработать переменные, но «не знает», где их взять;
- итератор выполняет алгоритм обработки структуры данных в соответствии со своим назначением: **foreach** - обходит все переменные, **firstthat** - обходит и проверяет все переменные, итератор сортировки - сортирует указатели на хранимые объекты (или соответствующие элементы структуры данных, например, элементы списка);

- действие, которое надлежит выполнить над хранимыми объектами произвольного типа, реализуется вызовом внешней функции через указатель. Итератор извлекает из структуры данных указатели **void*** на хранимые данные и передает их внешней функции.
- прототип (схема формальных параметров и результата) внешней функции различна для разных типов итераторов. Для **foreach** – указатель имеет вид **void (*pf)(void*)**, для **firstthat** – **int (*pf)(void*)**, для итераторов, использующих сравнение – **int (*cmp)(void*,void*)**.

```
//-----93-04.cpp
//---- Итераторы foreach, firstthat и поиска минимального для списка
struct list { list *next;           // Указатель на следующий
void *pdata; };                   // Указатель на данные
//---- Итератор: для каждого элемента списка
void ForEach(list *pv, void (*pf)(void*) ) {
for (; pv !=NULL; pv = pv->next)
    (*pf)(pv->pdata);
}
//---- Итератор: поиск первого в списке по условию
void *FirstThat(list *pv, int (*pf)(void*)) {
for (; pv !=NULL; pv = pv->next)
    if ((*pf)(pv->pdata)) return pv->pdata;
return NULL; }
//---- Итератор: поиск минимального в списке
void *FindMin(list *pv, int (*pf)(void* ,void*))
{ list *pmin;
for ( pmin=pv; pv !=NULL; pv = pv->next)
    if ((*pf)(pv->pdata ,pmin->pdata) <0) pmin=pv;
return pmin; }
//---- Примеры использования итератора -----
//---- Функция вывода строки
void print(void *p) { puts((char*)p); }
//---- Функция проверки : длины строки >5
int bigstr(void *p) { return strlen((char*)p ) > 5; }
//---- Функция сравнения строк по длине
int scmp(void *p1, void *p2)
{ return strlen((char*)p1)- strlen((char*)p2); }
//---- Вызов итераторов для статического списка,
// содержащего указатели на строки
list a1=NULL,"aaaa", a2={&a1,"bbbbbb"}, a3={&a2,"cccc"}, *PH=&a3;

//---- Итератор сортировки для массива указателей
void Sort(void **pp, int (*pf)(void*,void*))
{ int i,k;
do
for (k=0,i=1; pp[i] !=NULL; i++)
    if ( (*pf)(pp[i-1],pp[i])>=0)           // вызов функции сравнения
    { void *q;                             // перестановка указателей
k++; q = pp[i-1]; pp[i-1] = pp[i]; pp[i] = q;
}
while(k); }
// Пример вызова итератора сортировки для массива
// указателей на целые переменные
int cmp_int(void *p1, void *p2)
{ return *(int*)p1-*(int*)p2; }
int b1=5, b2=6, b3=3, b4=2;
void *PP[] = {&b1, &b2, &b3, &b4, NULL};
void main()
{ char *pp;
ForEach(PH,print);
pp = (char*) FirstThat(PH,bigstr);
```

```

if (pp !=NULL) puts(pp);
pp = (char*) FindMin(PH,scmp);
if (pp !=NULL) puts(pp);
Sort(PP,cmp_int);
for (int i=0; PP[i]!=NULL;i++) printf("%d ",*(int*)PP[i]);
puts("");}

```

Лабораторный практикум

Преобразовать функцию сортировки с использованием массивов **(b)**, списков **(6.3)**, деревьев **(8.4, 8.5)** в итератор. Проверить его работу на двух структурах данных содержащих указатели на различные типы (например, целые и строки). Массив преобразовать в массив указателей.

Головоломка

Результат здесь очевиден, потому что не может быть другим. Будет выведена строка "I'm foo" или значение 6. Значительно труднее объяснить:

- что такое **P** - функция, указатель на функцию? Если функция, то где у нее определения формальных параметров и результата и что она делает, если указатель - то где обращение по нему;
- где находится операция, по которой на самом деле производится вызов функции **foo** и **inc**.

```

//-----93-05.cpp
//----- 1
void ( *P1(void(*ff)(void)))(void) { return ff; }
void foo1(void){ printf("I'm foo\n"); }
void main1(){(*P1(foo1))();}
//----- 2
int ( *P2(int(*ff)(int))(int) { return ff; }
int inc2(int n){ return n+1; }
void main2(){ printf("%d\n",(*P2(inc2))( 5 ));}
//----- 3
typedef void (*PF3)(void);
PF3 P3(PF3 ff) { return ff; }
void foo3(void){ printf("I'm foo\n"); }
void main3(){(*P3(foo3))();}
//----- 4
typedef int (*PF4)(int);
PF4 P4(PF4 ff) { return ff; }
int inc4(int n){ return n+1; }
void main4(){ printf("%d\n",(*P4(inc4))( 7 ));}

```